

| Tratamento De Erros

João Eduardo Montandon

DCC024

```
public int pop() {  
    Node n = top;  
    top = n.getLink();  
    return n.getData();  
}
```

O que poderia dar errado com esse método? 🤔

E se a pilha estiver vazia?

E se o programa ficar sem espaço de memória?

| Os Tratamentos Disponíveis

1. Pré-condições (Guardas)
2. Definição total
3. Erros Fatais
4. Flag de Erro
5. Tratamento de Exceção

| Pré-condições (Guardas)

Implementação de pré-condições que, uma vez satisfeitas, executam o código sem ocorrência de erro.

```
public static void main(String[] args) {  
    IntStack s = new Stack();  
    if (!s.isEmpty())  
        s.pop();  
}
```

- Comportamentos erráticos devem ser documentados para tratamento.
- Torna o *caller* responsável pela verificação e interpretação do erro.

| Definição Total

Definição interna de comportamentos "esperados" para situações de erro.

```
public int pop() {  
    Node n = top;  
    if (n == null)  
        return 0;  
  
    top = n.getLink();  
    return n.getData();  
}
```

Estratégia bem utilizada em algumas circunstâncias.

```
System.out.println(0.0/0.0);
```

Qual será o resultado disso? 🤔

- Outros casos: I/O (**EOF**), conversão (**atoi()**), etc...
- Porém, pode mascarar o problema para quem utilizar a função.

| Erro Fatal

Aborta o programa arbitrariamente caso um erro seja detectado.

```
public int pop() {
    Node n = top;
    if(n == null) {
        System.out.println("Erro: desempilhando
pilha vazia!");
        System.exit(-1);
    }
    top = n.getLink();
    return n.getData();
}
```

- Oposto a definição total.
- Tem impacto global (encerramento do programa).
- Não é boa para programas orientado a objetos.
 - O impacto de um objeto deveria ficar restrito ao seu escopo.

| Flag De Erro

Uso de um valor ou variável especial para indicar que o processamento falhou.

```
public class IntStack {
    public int top() {
```

```

        Node n = top;
        if(n == null) {
            this.error = true;
            return 0;
        }
        ...
    }
    public boolean getError() { return error; }
    public void resetError() { error = false; }
}

```

```

public static void main(String[] args) {
    IntStack s = new IntStack();
    s.add(1);
    s.pop();
    s.pop();
    if(s.getError())
        System.out.println("Erro!");
}

```

- Libera o valor utilizado como indicação de erro [anteriormente](#).
- O *caller* precisa verificar explicitamente o erro, **sempre**.

| Tratamento De Exceções

- Uma exceção é um erro que impede o fluxo de uma computação.
- Em linguagens mais antigas, exceções eram fatais e levavam a quebra do programa.
- Linguagens modernas permitem capturar a exceção para recuperar a execução do programa.

- Antes: Terminavam o fluxo de execução do programa.
- Hoje: Alternativas ao fluxo de execução principal do programa.

```
public class TestException {  
    public static void main(String[] args) {  
        int i = Integer.parseInt(args[0]);  
        int j = Integer.parseInt(args[1]);  
        System.out.println(i / j);  
    }  
}
```

Qual o fluxo principal? 🤔

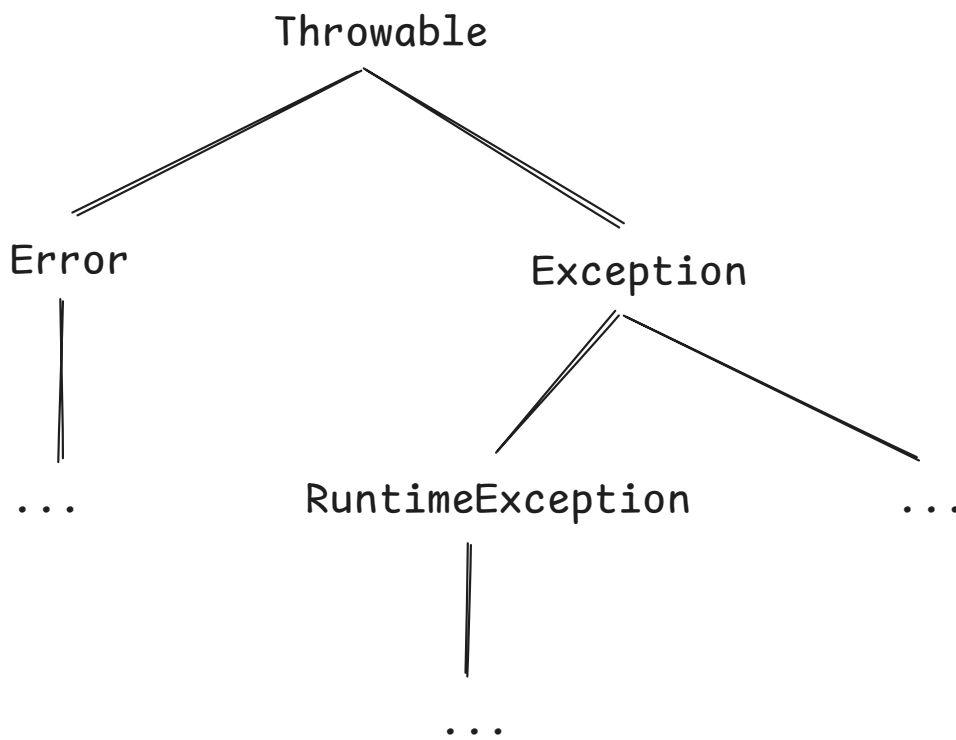
Quais os fluxos alternativos? 🤔

```
$ java TestException 6 3  
2
```

```
$ java TestException 6 0  
Exception in thread "main" java.lang.ArithmeticException:  
...
```

```
$ java TestException  
Exception in thread "main"  
java.lang.ArrayIndexOutOfBoundsException: ...
```

| Hierarquia



- Error: Erros gerados a partir do sistema.
 - `StackOverflowError`, `OutOfMemoryError`, etc.
- Exception: Erros com origem no próprio programa.
 - `NullPointerException`, `ArithmeticException`, etc.

Captura De Exceções

```
<try> ::= <try> <catch>  
<catch> ::= try <statements>  
<catch> ::= catch (<type> <var-name>) <statements>
```

1. Comandos são executados no bloco `try`
2. Se não houver exceção, `catch` não é executado.
3. Se a exceção lançada é do tipo `type`, executa-se o bloco do `catch`.

```
try {  
    int i = Integer.parseInt(args[0]);  
    int j = Integer.parseInt(args[1]);  
    System.out.println(i / j);  
} catch (ArithmeticException e) {  
    System.out.println("Divisão por zero!");  
}
```

Mas e quanto ao `ArrayIndexOutOfBoundsException`? 🤔

Opção 1: Declarar exceções mais genéricas.

```
try {  
    int i = Integer.parseInt(args[0]);  
    int j = Integer.parseInt(args[1]);  
    System.out.println(i / j);  
} catch (RuntimeException e) {  
    System.out.println("Erro!");  
    e.printStackTrace();  
}
```

Opção 2: Encadear blocos de `catch`.

```
try {  
    int i = Integer.parseInt(args[0]);  
    int j = Integer.parseInt(args[1]);  
    System.out.println(i / j);  
} catch (ArithmeticException e) {  
    System.out.println("Divisão por zero!");  
} catch (ArrayIndexOutOfBoundsException e) {  
    System.out.println("Requer dois parâmetros!");  
}
```

- Apenas um dos blocos `catch` é executado.
- Todos os blocos `catch` devem ser alcançáveis.

| Fluxo

```
System.out.print("1, ");
try {
    String s = null;
    s.length();
} catch (NullPointerException e) {
    System.out.print("2, ");
}
System.out.println("3");
```

O que vai ser impresso nesse programa? 😬

- O bloco `try/catch` é apenas outro comando em Java.
- O comando após o bloco será executado tanto se não houver erros, quanto se o erro for tratado pelo bloco `catch`.
- Indiretamente, **permite recuperar o estado do programa** para após o erro.

| Escopo

```
public static void g() { System.out.println(6/0); }
public static void f() {
    try {
        g();
    } catch (ArithmeticException e) {
```



```
        System.out.println("Erro!");
    }
}
```

O que esse programa mostrará na tela? 🤔

- Blocos `try` monitoram o período de ativação de todo seu bloco.
- Isso inclui chamadas a outros métodos dentro do bloco.
- Na verdade, o lançamento e captura podem estar distantes indefinidamente.
 - Rastreio é feito a partir dos [Registros De Ativação](#) presentes no *stack frame*.

```
public class ExceptionScopeFlow {
    public static void g() {
        System.out.println("Start g");
        int a = 10 / 0;
        System.out.println("Finish g");
    }
    public static void f() {
        System.out.println("Start f");
        try {
            g();
        } catch (RuntimeException e) {
            System.out.println("Error f");
        }
        System.out.println("Finish f");
    }
    public static void main(String[] args) {
        System.out.println("Start Main");
        f();
        System.out.println("Finish Main");
    }
}
```

O que vai ser impresso nesse programa? 🤔

| Lançamento De Exceção

- Uma exceção pode ser lançada arbitrariamente no programa.
- Permite integrar os erros do software ao mesmo mecanismo de tratamento para erros do sistema.

```
throw <EXPRESSAO>;
```

<EXPRESSAO> precisa referenciar um objeto derivado de `Throwable`.

```
public class OutOfQuotaException extends Exception {
    private int amount;
    public OutOfQuotaException(String msg, int amount) {
        super(msg);
        this.amount = amount;
    }
    public int getAmount() {
        return this.amount;
    }
}
```

```
try {
    throw new OutOfQuotaException("Quota excedida!",
20);
} catch(OutOfQuotaException ex) {
    System.out.println(ex.getMessage());
}
```

```
        System.out.println(ex.getAmount());  
    }
```

| Exceções Verificáveis

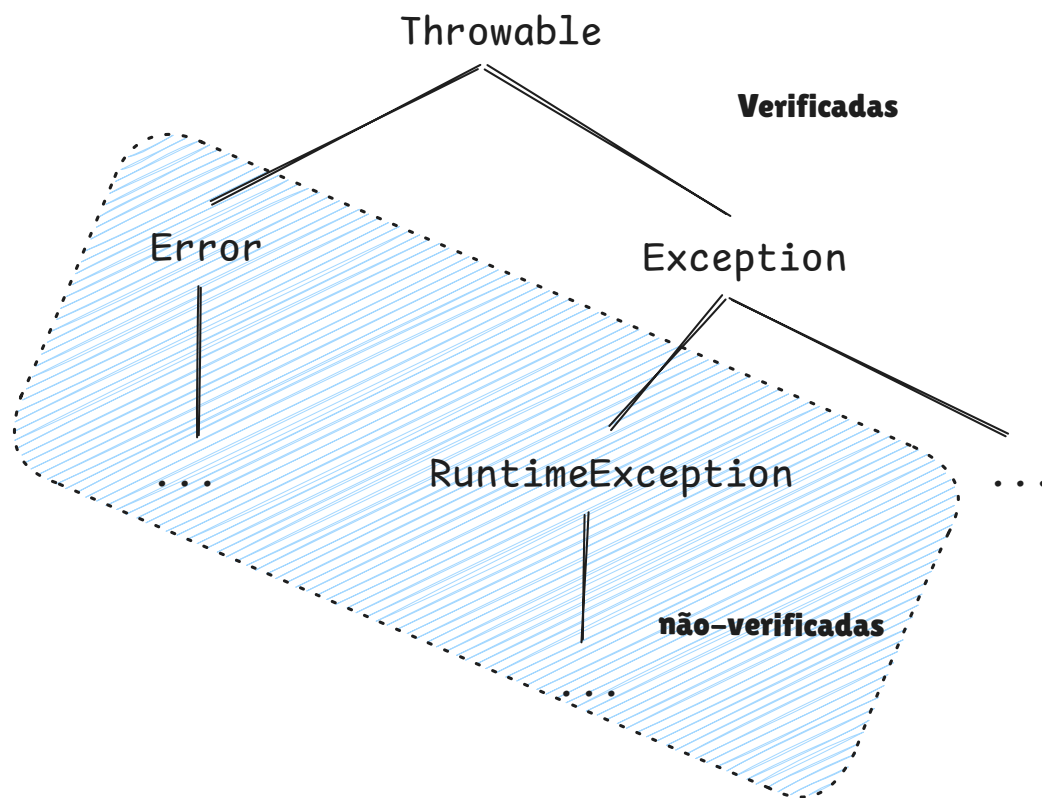
```
void f() {  
    throw new NullPointerException();  
}
```

Este código compila?

```
void f() {  
    throw new OutOfQuotaException("Quota excedida.",  
5);  
}
```

E este, compila? 🤔

- Java divide as exceções em dois grupos: verificadas e não-verificadas.
 - *Não-verificadas*: Herdam de `RuntimeException` ou `Error`
 - *Verificadas*: As demais que herdam de `Exception` / `Throwable`.
- Exceções verificadas **precisam** ser tratadas no programa.



Exceções podem ser tratadas de duas formas:

1. **try/catch**: captura e finaliza exceção que foi lançada.
2. **throws**: propaga a exceção para a próxima chamada no *stack frame*.

```
void f() throws OutOfQuotaException {
    throw new OutOfQuotaException("Quota excedida.",
5);
}

public static void main(String[] args) {
    try {
        f();
    } catch(OutOfQuotaException ex) {
        System.out.println(ex.getMessage());
        System.out.println(ex.getAmount());
    }
}
```

```
}  
}
```

| Verificada Ou Não-verificada?

- Uma exceção verificada **não pode ser ignorada**.
- Tenha certeza de que o seu lançamento implica em um tratamento, sempre.
- Exceção é levantada em vários lugares, mas tratada poucas vezes: *não-verificada*.
- Exceção é levantada poucas vezes, mas deve ser sempre tratada: *verificada*.

| Bloco Finally

Bloco que será sempre executado quando o `try/catch` é finalizado, *independentemente do que ocorrer*.

```
try {  
    ...  
} catch (Exception1 e) {  
    ...  
} catch (Exception2 e) {  
    ...  
} finally {  
    ...  
}
```

O que vai ser impresso aqui? 🤔

```
System.out.print("1");  
try {  
    System.out.print("2");
```

```
        if (true) throw new Exception();  
        System.out.print("3");  
    } catch (Exception e) {  
        System.out.print("4");  
    } finally {  
        System.out.print("5");  
    }  
    System.out.print("6");
```

| Considerações Finais

- O tratamento de exceção permite a um programa retornar ao seu estado consistente antes de ser encerrado.
- Presente na grande maioria das linguagens.